BACKEND ENGINEER INTERVIEW MASTER GUIDE

FAANG · Product Companies · SDE-2 / Senior Roles

Java · Spring Boot · DSA · System Design · AWS · Kafka · SQL · Microservices

Compiled: 24 May 2026 | Version 1.0 | 6-Month Prep Edition

1. Skill Gap Analysis & Priority Roadmap

Where you stand · What to focus on · How to sequence

1.1 Current Skill Gap Analysis

Your Profile: 3.4 years Backend Engineer — Java, Spring Boot, Kafka, REST, AWS, CI/CD

Domain	Current Level	Target Level	Why It Matters	Priority
DSA & Problem Solving	Beginner–Med	FAANG-Ready	CRITICAL — 70% hiring filter	HIGH
Java Core / Advanced	Intermediate	Advanced	HIGH — every round has Java Qs	HIGH
Spring Boot / Microservices	Intermediate+	Advanced	HIGH — arch & coding rounds	HIGH
System Design (LLD+HLD)	Beginner–Int	SDE-2 Level	CRITICAL — dedicated round at all cos	HIGH
AWS Backend Engineering	Intermediate	Advanced	MED-HIGH — infra + scenario Qs	MED
SQL & DB Optimization	Intermediate	Advanced	HIGH — always tested	MED
Kafka / Event-Driven	Intermediate	Advanced	HIGH — you have exp, deepen it	MED
Docker / Kubernetes	Beginner	Working know.	MED — conceptual at SDE-2	LOW

1.2 Priority Stack Ranking

Rank	Topic	Interview Weight	Weeks to Invest
1	DSA (Arrays, Graphs, DP, Trees, Sliding Win)	35–40%	Throughout all 6 months

2	System Design — HLD + LLD	25–30%	Weeks 5–24 (continuous)
3	Java Advanced (JVM, Threads, Collections)	20%	Weeks 2–6
4	Spring Boot & Microservices Internals	15%	Weeks 4–10
5	Kafka & Event-Driven Architecture	10%	Weeks 8–12
6	SQL & DB Optimization	10%	Weeks 6–10
7	AWS Backend Engineering	10%	Weeks 10–14
8	Secondary (K8s, Redis, Docker, Security)	5%	Weeks 16–20

2. 6-Month Optimized Study Plan

Week-by-week execution blueprint

2.1 Phase Overview

Phase	Weeks	Theme	Key Topics
Phase 1	Weeks 1–4	Foundation Sprint	DSA Basics + Java Core + Spring Refresher
Phase 2	Weeks 5–10	Core Build	DSA Medium + Java Advanced + Spring Deep + SQL
Phase 3	Weeks 11–16	System Design	HLD + LLD + Kafka + AWS + Design Patterns
Phase 4	Weeks 17–20	Advanced Topics	DSA Hard + Microservices Patterns + AWS Deep
Phase 5	Weeks 21–24	Interview Mode	Mock Interviews + Revisions + Target Company Prep

2.2 Weekly Breakdown

Phase 1 — Foundation Sprint (Weeks 1–4)

Week	Focus
Week 1	Arrays, Strings, Two Pointers, Sliding Window (DSA) + JVM Internals, GC, Memory (Java)
Week 2	Linked Lists, Stacks, Queues, Hashing (DSA) + Multithreading, synchronized, volatile (Java)
Week 3	Binary Search, Recursion, Fast & Slow Pointers (DSA) + Spring IoC, Bean lifecycle, DI (Spring)
Week 4	Trees (BFS/DFS), Heaps (DSA) + Executors, CompletableFuture, Streams (Java) + JPA basics

Phase 2 — Core Build (Weeks 5–10)

Week	Focus
Week 5	Graphs (BFS/DFS/Dijkstra) + HashMap internals, ConcurrentHashMap, ThreadLocal (Java)
Week 6	Tries, Union-Find, Intervals (DSA) + SQL joins, indexes, EXPLAIN, query optimization
Week 7	Greedy, Backtracking (DSA) + Spring Security, JWT, OAuth2, exception handling (Spring)
Week 8	DP Introduction — 1D DP patterns (DSA) + Kafka producer/consumer, partitions, offsets
Week 9	DP Advanced — 2D DP, Knapsack, LCS (DSA) + Transactions, Hibernate N+1, caching
Week 10	Bit Manipulation, Math (DSA) + Resilience patterns: Circuit Breaker, Retry, Bulkhead

Phase 3 — System Design (Weeks 11–16)

Week	Focus
Week 11	HLD: Scalability, CAP theorem, Load Balancing, Caching (Redis), CDN
Week 12	HLD: Design URL Shortener, Paste Bin, Rate Limiter
Week 13	HLD: Design Twitter Feed, Instagram, Notification System
Week 14	HLD: Design Uber/Food Delivery, Payment System + AWS deep (EC2, RDS, S3, Lambda)
Week 15	LLD: Design Parking Lot, Library, BookMyShow + SOLID + Design Patterns
Week 16	LLD: Design Chess, Elevator, Snake & Ladder + API design + Microservices patterns

Phase 4 — Advanced (Weeks 17–20)

Week	Focus
Week 17	DSA Hard patterns: Segment Trees, Advanced DP, Monotonic Stack/Queue
Week 18	Microservices: Service Mesh, Saga pattern, CQRS, Event Sourcing, Distributed Tracing
Week 19	AWS: ECS/EKS, API Gateway, CloudWatch, CI/CD pipelines, IAM deep-dive
Week 20	Kafka deep: Kafka Streams, exactly-once semantics, rebalancing, consumer groups

Phase 5 — Interview Mode (Weeks 21–24)

Week	Focus
Week 21	Company-specific prep: Flipkart/Walmart style — DSA (Graphs, DP) + HLD (e-commerce)
Week 22	Company-specific prep: Razorpay/PhonePe — Payments System Design + Java concurrency
Week 23	Company-specific prep: Swiggy/Zomato/Uber — Geo-distributed systems, real-time tracking
Week 24	Full mock weeks: 2 mock interviews/day + rapid revision of all cheatsheets

3. Daily Execution Plan

How to structure every single day for maximum output

3.1 Weekday Schedule (3–4 Hours/day)

Time Slot	Duration	Activity	Notes
6:00–6:30 AM	30 min	DSA Problem — Revision (yesterday's)	Re-solve without looking
6:30–7:30 AM	60 min	New DSA Problem (1 Medium)	Think aloud, then code
Evening 6–7 PM	60 min	Theory topic (Java/Spring/SD)	Active reading + notes
Evening 7–8 PM	60 min	Concept coding / Deep dive	Implement from scratch
Evening 8–8:30 PM	30 min	Flashcard review (Anki)	Spaced repetition

3.2 Weekend Schedule (6–7 Hours/day)

Time Slot	Duration	Activity
9:00–11:00 AM	2 hrs	Timed DSA contest / 3 problems back to back
11:00–12:30 PM	90 min	System Design — design one system end to end
1:30–3:00 PM	90 min	Deep topic study (JVM / Kafka / AWS)
3:00–4:30 PM	90 min	Mock interview (with peer / solo simulation)
4:30–5:00 PM	30 min	Weekly revision — all flashcards

3.3 The Daily DSA Framework

- **UNDERSTAND (5 min):** Read problem → Identify input/output → Note constraints → Ask clarifying questions
- **BRUTE FORCE (5 min):** Think of naive solution → State time/space → Never code brute force in interview
- **OPTIMIZE (10 min):** Identify bottleneck → Apply pattern (sliding window / two pointer / BFS etc.)
- **CODE (15 min):** Write clean code → Use meaningful variable names → Handle edge cases
- **TEST (5 min):** Trace through with examples → Check edge cases: empty, single element, negatives, overflow

4. DSA Roadmap

Pattern-based problem solving · FAANG-curated problems · Revision strategy

4.1 DSA Pattern Master Map

#	Pattern	Difficulty	Key Problems	LeetCode
1	Sliding Window	Medium	Max sum subarray, Longest substring without repeat, Minimum window substring	LC 3, 76, 209, 424
2	Two Pointers	Easy-Med	Pair sum, Dutch flag, Container with most water, Trapping rain water	LC 11, 15, 42, 167
3	Fast & Slow Pointers	Easy-Med	Detect cycle, Find middle, Floyd's cycle detection	LC 141, 142, 202, 876
4	Merge Intervals	Medium	Merge intervals, Insert interval, Non-overlapping intervals	LC 56, 57, 435, 986
5	Cyclic Sort	Easy-Med	Missing number, Find duplicate, First K missing positives	LC 268, 287, 448
6	In-place Reversal (LinkedList)	Medium	Reverse LL, Reverse sub-list, Reverse every K group	LC 92, 206, 25
7	BFS — Trees/Graphs	Medium	Level order, Zigzag, Min depth, Shortest path, Rotten oranges	LC 102, 103, 127, 994
8	DFS — Trees/Graphs	Medium	Path sum, All paths, Number of islands, Clone graph	LC 200, 133, 112, 257
9	Two Heaps	Hard	Median finder, Sliding window median, IPO scheduling	LC 295, 480, 502
10	Subsets / Backtracking	Med-Hard	All subsets, Permutations, Combination sum, N-Queens	LC 78, 46, 39, 51
11	Modified Binary Search	Medium	Rotated array, Find peak, Search in 2D matrix, Koko eating	LC 33, 153, 74, 875
12	Top K Elements	Medium	Top K frequent, K closest points, Sort K-sorted array	LC 347, 973, 215
13	K-way Merge	Hard	Merge K sorted lists, Smallest range covering K lists	LC 23, 632
14	Dynamic Programming	Med-Hard	Fibonacci, Knapsack, LCS, Edit distance, Coin change, House robber	LC 70, 198, 300, 322, 1143
15	Topological Sort (DAG)	Medium	Course schedule, Alien dictionary, Task scheduling	LC 207, 210, 269
16	Union Find (DSU)	Medium	Number of connected components, Redundant connection, Accounts merge	LC 323, 684, 721
17	Trie	Medium	Implement Trie, Word search II, Replace words, Auto-complete	LC 208, 212, 648
18	Monotonic Stack	Medium	Next greater element, Daily temperatures, Largest rectangle in histogram	LC 84, 496, 739
19	Graph Algorithms	Hard	Dijkstra, Bellman-Ford, Floyd-Warshall, Prim's MST	LC 743, 787, 1514
20	Bit Manipulation	Easy-Med	Single number, Power of 2, Bit counting, XOR tricks	LC 136, 137, 191, 231

4.2 Curated 100 DSA Problems — By Difficulty

Easy (20 problems) — Warm-up, never skip

LC#	Problem	Pattern	Key Insight
LC 1	Two Sum	HashMap	Foundation of all hash-map problems
LC 121	Best Time Buy/Sell Stock	Greedy	One-pass O(n)
LC 206	Reverse Linked List	Pointers	Master iterative + recursive
LC 141	Linked List Cycle	Fast-Slow	Floyd's algorithm
LC 53	Maximum Subarray	Kadane's	Classic DP / Greedy
LC 20	Valid Parentheses	Stack	Always asked
LC 169	Majority Element	Boyer-Moore	O(1) space trick
LC 543	Diameter of Binary Tree	DFS	Global max pattern
LC 226	Invert Binary Tree	DFS/BFS	Classic recursion
LC 104	Max Depth of Binary Tree	DFS	Tree recursion base
LC 217	Contains Duplicate	HashSet	Warm-up
LC 383	Ransom Note	HashMap	Frequency count
LC 242	Valid Anagram	Sorting/Map	Sorting vs hash tradeoff
LC 704	Binary Search	Binary Search	Template to memorize
LC 875	Koko Eating Bananas	Binary Search	Binary search on answer
LC 268	Missing Number	Math/XOR	Multiple approaches
LC 191	Number of 1 Bits	Bit Manip	n & (n-1) trick
LC 70	Climbing Stairs	DP	Fibonacci foundation
LC 338	Counting Bits	DP+Bits	DP on bits
LC 572	Subtree of Another Tree	DFS	Serialization approach

Medium (60 problems) — Core of every FAANG round

LC#	Problem	Pattern	Key Insight
LC 3	Longest Substring Without Repeat	Sliding Window	HashSet + expand/contract
LC 76	Minimum Window Substring	Sliding Window	Hard variant — frequency map
LC 11	Container With Most Water	Two Pointers	Greedy shrink from max height
LC 15	3Sum	Two Pointers	Sort + fix one + two pointer

LC 42	Trapping Rain Water	Two Pointers	Max-left, max-right from both ends
LC 238	Product of Array Except Self	Prefix/Suffix	No division trick
LC 56	Merge Intervals	Sort+Merge	Classic interval — memorize
LC 57	Insert Interval	Binary Search	Variant of merge
LC 435	Non-overlapping Intervals	Greedy	Sort by end time
LC 146	LRU Cache	LinkedHashMap	Design — must know internals
LC 380	Insert Delete GetRandom O(1)	HashMap+List	Classic design
LC 200	Number of Islands	BFS/DFS/DSU	3 approaches — master all
LC 695	Max Area of Island	DFS	Variation of 200
LC 286	Walls and Gates	Multi-source BFS	BFS from all gates
LC 994	Rotting Oranges	Multi-src BFS	Classic multi-source
LC 207	Course Schedule	Topo Sort	Cycle detection in directed graph
LC 210	Course Schedule II	Topo Sort	Return order too
LC 417	Pacific Atlantic Water Flow	Reverse DFS	Think backwards
LC 127	Word Ladder	BFS	Level BFS for shortest path
LC 23	Merge K Sorted Lists	Heap	PriorityQueue
LC 347	Top K Frequent Elements	Heap/Bucket	Bucket sort O(n)
LC 295	Find Median from Data Stream	Two Heaps	Max-heap + min-heap
LC 33	Search in Rotated Sorted Array	Binary Search	Check which half sorted
LC 153	Find Min in Rotated Array	Binary Search	Modified BS
LC 875	Koko Eating Bananas	Binary Search	BS on answer space
LC 102	Binary Tree Level Order	BFS	Template for all BFS trees
LC 199	Binary Tree Right Side View	BFS/DFS	BFS — last in level
LC 105	Construct BT from Pre+Inorder	DFS	Classic tree construction
LC 235	LCA of BST	BST Property	Simple traversal
LC 236	LCA of Binary Tree	DFS	Post-order traversal
LC 98	Validate BST	DFS+Bounds	Pass min/max bounds
LC 230	Kth Smallest in BST	In-order	Morris traversal bonus
LC 208	Implement Trie	Trie	Build TrieNode class
LC 211	Design Add & Search Words	Trie+DFS	Wildcard in trie
LC 190	Reverse Bits	Bit Manip	Shift and OR
LC 371	Sum of Two Integers	Bit Manip	No + operator

LC 78	Subsets	Backtracking	Bit mask + BT approaches
LC 46	Permutations	Backtracking	Swap approach
LC 39	Combination Sum	Backtracking	Allow reuse
LC 40	Combination Sum II	Backtracking	No reuse + skip duplicates
LC 90	Subsets II	Backtracking	Handle duplicates
LC 17	Letter Combinations Phone	Backtracking	Mapping pattern
LC 131	Palindrome Partitioning	BT + DP	Check palindrome with DP
LC 198	House Robber	DP	1D DP foundation
LC 213	House Robber II	DP	Two passes — circular
LC 139	Word Break	DP	Bottom-up DP
LC 300	Longest Increasing Subsequence	DP/BinSrch	O(n log n) with patience sort
LC 322	Coin Change	DP	Classic bottom-up knapsack
LC 518	Coin Change II	2D DP	Count ways — unbounded knapsack
LC 1143	Longest Common Subsequence	2D DP	Foundation of edit distance
LC 62	Unique Paths	2D DP	Grid DP — O(n) space
LC 64	Minimum Path Sum	2D DP	Modify in-place
LC 91	Decode Ways	DP	Check 1 and 2 digit combos
LC 72	Edit Distance	2D DP	Classic Levenshtein
LC 739	Daily Temperatures	Mono Stack	Next greater element
LC 84	Largest Rectangle Histogram	Mono Stack	Hard but very asked
LC 560	Subarray Sum Equals K	Prefix Sum	Prefix + HashMap
LC 525	Contiguous Array	Prefix Sum	0/1 balance trick
LC 253	Meeting Rooms II	Heap/Sweep	Min heap for end times
LC 31	Next Permutation	Two Pointers	In-place algorithm

Hard (20 problems) — Differentiator for top offers

LC#	Problem	Pattern
LC 4	Median of Two Sorted Arrays	Binary Search
LC 23	Merge K Sorted Lists	Heap
LC 25	Reverse Nodes in K-Group	Linked List
LC 51	N-Queens	Backtracking

LC 76	Minimum Window Substring	Sliding Window
LC 84	Largest Rect in Histogram	Monotonic Stack
LC 85	Maximal Rectangle	Stack + DP
LC 124	Binary Tree Max Path Sum	DFS
LC 297	Serialize/Deserialize BT	BFS/DFS
LC 239	Sliding Window Maximum	Deque/Mono Q
LC 295	Find Median from Data Stream	Two Heaps
LC 315	Count Smaller After Self	Merge Sort/BIT
LC 329	Longest Inc Path in Matrix	DFS + Memo
LC 336	Palindrome Pairs	Trie/HashMap
LC 354	Russian Doll Envelopes	DP + BinSrch
LC 42	Trapping Rain Water	Two Pointers
LC 480	Sliding Window Median	Two Heaps
LC 632	Smallest Range K Lists	Heap + Sliding W
LC 685	Redundant Connection II	Union-Find
LC 815	Bus Routes	BFS on sets

4.3 DSA Revision Strategy

> 🔁 After solving: tag each problem with pattern + difficulty. Every Sunday, re-solve 5 problems from the week without looking at solutions.

- **Week 1–4:** Solve 1 easy + 1 medium daily. Focus on understanding, not speed.
- **Week 5–12:** Solve 2 mediums daily. Start timing yourself (25 min per medium).
- **Week 13–20:** 1 medium + 1 hard every 2 days. LeetCode weekly contests.
- **Week 21–24:** Timed mock: 3 problems in 60 min. Full interview simulation.

4.4 Common DSA Mistakes in Interviews

- ❌ **Jumping to code immediately:** Always state brute force first, then optimize. Interviewers penalize silent coding.
- ❌ **Not handling edge cases:** Empty input, null, single element, negative numbers, integer overflow.
- ❌ **Using wrong data structure:** Arrays when you need O(1) lookup → use HashMap. Lists when you need O(1) insert → use Deque.
- ❌ **Ignoring time complexity:** Always state TC and SC. Know when O(n log n) is acceptable vs O(n).
- ❌ **Over-engineering:** A clean O(n) solution beats a buggy O(log n) attempt.

5. Java Advanced Roadmap

JVM Internals · Concurrency · Collections · GC · Performance

5.1 JVM Internals — Must Know for FAANG

JVM Memory Architecture

Memory Area	What's Stored	Thread Safe?	GC Managed?	Interview Tip
Heap	Objects, instance variables	No	Yes	Where GC happens; tuned with -Xmx
Stack	Method frames, local vars, references	Yes (per thread)	No	StackOverflow = deep recursion
Method Area / Metaspace	Class metadata, static fields, bytecode	No	Partially	PermGen → Metaspace in Java 8
PC Register	Current instruction pointer	Yes (per thread)	No	One per thread
Native Method Stack	JNI calls to native OS methods	Yes (per thread)	No	Used by JNI code

Garbage Collection — Deep Dive

- **Minor GC:** Collects Young Generation (Eden + S0 + S1). Stop-the-world but brief. Objects surviving N GC cycles promoted to Old Gen.
- **Major/Full GC:** Collects Old Generation. Long pause. Aim to minimize in production — tune with -XX:NewRatio
- **G1GC (Java 9+ default):** Region-based GC. Predictable pause times. -XX:MaxGCPauseMillis=200
- **ZGC / Shenandoah:** Low-latency GC. Sub-millisecond pauses. Used in latency-sensitive microservices (PhonePe, Razorpay style)

5.2 Java Concurrency — Most Asked Interview Topics

Key Concurrency Concepts

Concept	What It Does	Use Case	Gotcha
synchronized	Mutual exclusion on a monitor	Single-JVM simple locking	Blocks all other threads — performance hit
volatile	Ensures visibility across threads (no caching)	Flag variables, simple pub-sub	NOT atomic for compound ops (i++ is not thread-safe)
ReentrantLock	Explicit lock with try/finally, fairness	When you need timed/conditional locking	Always unlock in finally block
ReentrantReadWriteLock	Multiple readers OR one writer	High-read low-write scenarios	Writer starvation possible without fairness
AtomicInteger / AtomicReference	CAS-based lock-free ops	Counters, references in concurrent code	CAS loop can spin — not for high-contention

CountDownLatch	Wait until N operations complete	Service startup, parallel task completion	Cannot be reset — use CyclicBarrier for that
CyclicBarrier	Wait until N threads all arrive	Parallel computation phases	Can be reset unlike CountDownLatch
Semaphore	Limit concurrent access to N threads	Rate limiting, connection pools	Release in finally to avoid deadlock
ThreadLocal	Thread-confined variable storage	Request context, database connections	Memory leak in thread pools — always remove()
StampedLock	Optimistic read locking (Java 8+)	High-throughput read-dominated workloads	Complex API — validate stamp after optimistic read

Thread Pool & Executors — Must-Know

- **FixedThreadPool(n):** Fixed n threads. Unbounded queue. Risk: queue grows forever under load. Use for CPU-bound tasks.
- **CachedThreadPool:** Creates threads on demand, reuses idle ones. Risk: thread explosion under bursty load. Use for short-lived async tasks.
- **ScheduledThreadPool:** For periodic/delayed tasks. Replace Timer which swallows exceptions.
- **WorkStealingPool (Java 8+):** ForkJoinPool under the hood. Good for recursive divide-and-conquer.
- **Custom ThreadPoolExecutor:** ALWAYS prefer this in production: set corePoolSize, maxPoolSize, queue type (bounded!), rejection policy.

CompletableFuture — Advanced Async (Java 8+)

Method	Description	Example Use
supplyAsync()	Async computation returning value	Call external API asynchronously
thenApply()	Transform result (like map)	Parse response body
thenCompose()	Chain dependent futures (flatMap)	Sequential async calls
thenCombine()	Combine two independent futures	Fetch user + orders in parallel
allOf()	Wait for ALL futures to complete	Parallel fanout calls
anyOf()	Return first completed future	Hedged requests for latency
exceptionally()	Handle exception with fallback	Return cached value on failure
handle()	Handle both result and exception	Unified success/failure handler

5.3 Collections Internal Working

HashMap Internals — Most Asked Java Question

- **Internal Structure:** Array of Node<K,V>[] (buckets). Default capacity 16, load factor 0.75.
- **Hash Function:** key.hashCode() → spread with (h ^ (h >>> 16)) → bucket = hash & (n-1)
- **Collision Handling:** Chaining via LinkedList. Java 8: converts to Red-Black Tree when chain length > 8 (TREEIFY_THRESHOLD).
- **Resize:** When size > capacity * loadFactor → resize to 2x. All entries rehashed — O(n) cost!

- **HashMap vs ConcurrentHashMap:** HashMap: not thread-safe. ConcurrentHashMap: segment-level locking (Java 7) → CAS + synchronized on bucket head (Java 8). No full lock = better throughput.
- **Common Bug:** Using mutable objects as HashMap keys. If key's hashCode changes after insertion, the entry is lost.

5.4 Top 30 Java Interview Questions

#	Question	Key Answer Point
1	What is the difference between == and equals()?	== is reference compare; equals() is content compare. Override both equals() AND hashCode()
2	Explain String pool and intern()	String literals go to pool in Heap. intern() moves a new String to pool. Saves memory for repeated strings
3	What is the difference between Runnable and Callable?	Callable returns a value and can throw checked exceptions. Used with ExecutorService.submit()
4	Explain volatile vs synchronized	volatile: visibility only. synchronized: visibility + atomicity + ordering. Use volatile for simple flags
5	What is a deadlock? How to prevent it?	Two threads each hold a lock the other needs. Prevention: consistent lock ordering, lock timeout, tryLock()
6	What is the difference between HashMap and LinkedHashMap?	LinkedHashMap maintains insertion order (doubly-linked list of entries). Used for LRU cache
7	How does ConcurrentHashMap work in Java 8?	16 segments replaced by CAS + synchronized on individual bucket head. putIfAbsent() is atomic
8	What is the happens-before relationship?	JMM guarantee: actions before a write are visible to reads that see the write. Basis for volatile/sync
9	Explain ForkJoinPool and work stealing	Tasks split recursively. Idle threads steal tasks from other threads' deques. Basis of parallelStream()
10	What is the difference between Comparable and Comparator?	Comparable: natural order (compareTo in class). Comparator: external/multiple orderings. TreeSet, sort()
11	Explain Java Memory Model (JMM)	Defines how threads interact through shared memory. Core rules: visibility, ordering, atomicity guarantees
12	What is finalize() and why avoid it?	Called before GC. No guaranteed timing. Replaced by Cleaner/PhantomReference in Java 9+. Never rely on it
13	What are functional interfaces? Examples?	Single abstract method. Predicate, Function, Supplier, Consumer, BiFunction. Enable lambdas
14	How does Stream.parallel() work?	Uses ForkJoinPool.commonPool(). Splits data, processes in parallel, merges. Not always faster — overhead
15	What is Optional and when to use it?	Container that may or may not have a value. Use as return type, not parameter. Avoids NPE
16	Explain static vs instance variables in concurrency	Static: shared across all instances — synchronize access. Instance: one per object
17	What is a memory leak in Java? Example?	Objects retained in heap though not needed. Examples: static collections, unclosed streams, ThreadLocal not removed

18	What is the difference between fail-fast and fail-safe iterators?	Fail-fast (ArrayList): throws ConcurrentModificationException. Fail-safe (CopyOnWriteArrayList): iterates copy
19	Explain method references in Java 8	Shorthand for lambdas: Type::method. Static, instance, constructor refs. e.g. String::toUpperCase
20	What is CompletableFuture.thenCompose vs thenApply?	thenApply: sync transform (like map). thenCompose: async chaining (like flatMap) — avoids nested futures
21	What happens if a thread pool queue is full?	RejectedExecutionHandler fires. Policies: Abort, CallerRuns, Discard, DiscardOldest
22	What is a thread dump? How to take it?	Snapshot of all thread states. jstack <pid> or kill -3. Used to debug deadlocks, high CPU
23	Explain the difference between sleep() and wait()	sleep(): doesn't release lock. wait(): releases lock, used with notify()/notifyAll() for signaling
24	What is StampedLock? When to use it?	Optimistic read: read without acquiring lock, validate stamp afterward. Best for read-heavy, write-rare
25	How do you avoid race conditions in singleton?	Double-checked locking with volatile, or use enum singleton, or initialization-on-demand holder pattern
26	What is the difference between ArrayList and LinkedList for iteration?	ArrayList: O(1) random access. LinkedList: O(n) get. ArrayList wins for iteration due to cache locality
27	What is the G1 garbage collector?	Divides heap into equal regions. Collects regions with most garbage first. Predictable pause times
28	What is the difference between checked and unchecked exceptions?	Checked: must handle/declare (IOException). Unchecked: RuntimeException. Use unchecked for programming errors
29	What is the ClassLoader hierarchy?	Bootstrap → Extension/Platform → Application ClassLoader. Delegation model: child asks parent first
30	What is reflection? Performance implications?	Inspect/invoke classes at runtime. Bypasses compile-time checks. Slower than direct calls — cache Method objects

6. Spring Boot & Microservices

IoC · Beans · Security · JPA · Transactions · Resilience

6.1 Spring IoC Container & Bean Lifecycle

Bean Lifecycle (Exact Sequence — Asked in Interviews)

Step	Phase	What Happens
1	Instantiation	Spring creates bean instance via constructor/factory method
2	Populate Properties	Dependencies injected via setter/field injection

3	BeanNameAware.setBeanName()	Bean receives its name in the context
4	BeanFactoryAware / ApplicationContextAware	Bean gets reference to factory/context
5	BeanPostProcessor.postProcessBeforeInitialization()	Pre-init hooks (e.g., @Autowired processing)
6	@PostConstruct / InitializingBean.afterPropertiesSet()	Custom initialization logic
7	init-method (XML/annotation)	Additional init method if configured
8	BeanPostProcessor.postProcessAfterInitialization()	Post-init hooks (AOP proxy creation happens here)
9	Bean is Ready — In Use	Bean available in application context
10	@PreDestroy / DisposableBean.destroy()	Cleanup before bean removed from context

🔥 Interview Trick: AOP Proxies are created in step 8 (postProcessAfterInitialization). This is why you can't call @Transactional methods from within the same class — the proxy is bypassed.

6.2 Spring Security + JWT — Production Pattern

JWT Authentication Flow

- **1. Login:** POST /auth/login → UsernamePasswordAuthenticationToken → AuthenticationManager → UserDetailsService → Load user → Verify password → Generate JWT
- **2. Request:** Header: Authorization: Bearer <token> → JwtAuthenticationFilter → Extract + validate token → Set SecurityContext → Proceed to controller
- **3. Authorization:** @PreAuthorize("hasRole('ADMIN')") → SpEL expression evaluated against SecurityContext

JWT Part	Contains	Signed?
Header	Algorithm (HS256/RS256), token type	No
Payload	Claims: sub, iat, exp, roles, custom claims	No
Signature	HMACSHA256(base64(header) + '.' + base64(payload), secret)	Yes

❓ Common Interview Q: 'How do you invalidate a JWT?' → JWTs are stateless. Options: short expiry + refresh tokens, token blacklist in Redis, or opaque tokens with introspection.

6.3 Hibernate & JPA — Performance Traps

N+1 Problem — Most Common JPA Interview Q

- **Problem:** SELECT 1 query for parent list + N queries for each child collection. e.g., 100 orders → 101 queries!
- **Solution 1:** @EntityGraph or JOIN FETCH: SELECT o FROM Order o JOIN FETCH o.items — single query
- **Solution 2:** @BatchSize(size=20) — fetches related entities in batches
- **Solution 3:** Use projections (DTOs) — only fetch what you need

JPA Transaction Management

- **@Transactional:** Default propagation: REQUIRED (join existing or create new). Default isolation: READ_COMMITTED.
- **Propagation.REQUIRES_NEW:** Always creates new transaction. Parent transaction suspended. Use for audit logging — must succeed even if parent rolls back.
- **readOnly=true:** Hint to JPA to skip dirty checking. Improves performance for read-only operations.
- **Common Mistake:** Calling @Transactional method from same class bypasses Spring proxy. Use self-injection or restructure.

6.4 Top 30 Spring Boot Interview Questions

#	Question	Key Answer
1	What is the difference between @Component, @Service, @Repository?	All are @Component. @Service: business logic. @Repository: DB access + exception translation. Semantic difference
2	How does @Autowired work internally?	AutowiredAnnotationBeanPostProcessor scans beans. Resolves by type, then by @Qualifier name
3	What is the difference between @Bean and @Component?	@Bean: method-level in @Configuration, explicit instantiation. @Component: class-level, auto-detected
4	How does Spring handle circular dependencies?	Setter/field injection: allowed via early bean reference. Constructor injection: throws BeanCurrentlyInCreationException
5	What is Spring AOP? How does it work?	Aspect-Oriented. JDK Dynamic Proxy (interface) or CGLIB (class). Advice woven at postProcessAfterInitialization
6	What is @Transactional self-invocation issue?	Internal method calls bypass proxy. Fix: ApplicationContext.getBean() self-injection, or restructure to separate class
7	Explain @Cacheable, @CachePut, @CacheEvict	@Cacheable: returns cached. @CachePut: always executes + updates cache. @CacheEvict: removes from cache
8	What is Spring Boot autoconfiguration?	@EnableAutoConfiguration + spring.factories. Conditional beans loaded based on classpath, properties
9	How do you implement rate limiting in Spring Boot?	Bucket4j + Redis, or API Gateway (Kong/AWS) level. Custom filter with token bucket algorithm
10	What is the difference between @RestController and @Controller?	@RestController = @Controller + @ResponseBody. Serializes return to JSON automatically
11	How does exception handling work in Spring Boot?	@ControllerAdvice + @ExceptionHandler. GlobalExceptionHandler. Problem Details (RFC 7807)
12	What is the difference between EAGER and LAZY loading?	EAGER: loads relationship immediately. LAZY: loads on first access. LAZY = N+1 risk outside transaction
13	How do you implement async processing in Spring Boot?	@Async on method + @EnableAsync on config. Uses ThreadPoolTaskExecutor. Returns CompletableFuture
14	What is Resilience4j? How do you use it?	Circuit Breaker, Rate Limiter, Retry, Bulkhead. @CircuitBreaker(name='svc', fallbackMethod='fb')

15	How do you implement distributed caching with Redis?	spring-data-redis + @Cacheable. RedisTemplate for direct access. Pub/Sub for cache invalidation
16	What is the difference between monolith and microservices?	Monolith: single deployable. Micro: independent services per domain. Trade-off: operational complexity vs scalability
17	How do you handle service discovery in microservices?	Eureka (Netflix/Spring Cloud) or Kubernetes DNS or AWS ALB + Cloud Map
18	What is the Saga pattern?	Distributed transaction via sequence of local transactions. Choreography (events) vs Orchestration (central)
19	What is CQRS?	Command Query Responsibility Segregation. Separate read/write models. Event sourcing often paired
20	How do you implement distributed tracing?	Micrometer + Zipkin/Jaeger. Propagate trace-id, span-id in HTTP headers and Kafka message headers
21	What is the difference between FeignClient and RestTemplate?	FeignClient: declarative, integrates with Eureka/LB natively. RestTemplate: imperative, deprecated in Spring 6
22	How do you handle database migrations in microservices?	Flyway or Liquibase. Version-controlled scripts. Backward-compatible changes for zero-downtime deploys
23	What is Spring Cloud Gateway?	API Gateway built on Project Reactor. Route predicates, filters, rate limiting, authentication
24	How do you implement idempotency in REST APIs?	Client sends Idempotency-Key header. Store key + response in Redis with TTL. Return cached response on retry
25	What is the Outbox Pattern?	Write event to same DB table as business operation (same transaction). Separate process polls and publishes to Kafka
26	How do you secure inter-service communication?	mTLS, service mesh (Istio), JWT propagation, API keys for internal services
27	What is the Bulkhead pattern?	Isolate failures: separate thread pools per service. If one service is slow, doesn't consume all threads
28	How do you handle configuration in microservices?	Spring Cloud Config Server, AWS Parameter Store, HashiCorp Vault for secrets
29	What is eventual consistency?	In distributed systems, data will eventually be consistent. Acceptable for non-critical reads. Use events to sync
30	How do you do health checks in Spring Boot?	Actuator /actuator/health. Custom HealthIndicator. Kubernetes liveness vs readiness probes

7. System Design — HLD & LLD

Scalability · CAP · Caching · Sharding · Real-world architectures

7.1 System Design Interview Framework

> 📋 Every system design answer must follow: REQUIREMENTS → ESTIMATION → API DESIGN → HIGH-LEVEL DESIGN → DEEP DIVE → TRADE-OFFS. Don't jump to solution!

Step	Duration	What to Cover
1. Clarify Requirements	5 min	Functional (what it does) + Non-functional (scale, latency, consistency, availability)
2. Capacity Estimation	5 min	DAU, QPS (read/write), storage, bandwidth. Back-of-envelope math.
3. API Design	5 min	RESTful endpoints, request/response schema, pagination, versioning
4. High-Level Design	10 min	Components: clients, CDN, API Gateway, services, DB, cache, queue
5. Deep Dive	15 min	Most critical component. Data model, DB choice, sharding strategy, caching layer
6. Trade-offs	5 min	What you'd change at 10x scale. Consistency vs availability choices. Bottlenecks

7.2 CAP Theorem — Deep Understanding

Property	Definition	Example Systems
Consistency	Every read sees the most recent write (or an error)	RDBMS, ZooKeeper, HBase
Availability	Every request gets a (non-error) response (may be stale)	Cassandra, DynamoDB, CouchDB
Partition Tolerance	System works despite network partition between nodes	Required for any distributed system

- **CP systems** (Consistency + Partition): Return error when partitioned. Bank ledger, inventory management.
- **AP systems** (Availability + Partition): Return potentially stale data. Social feeds, shopping carts.

> 💬 Interviewers love: 'We chose AP over CP here because users seeing slightly stale feed is acceptable. We'll reconcile with eventual consistency via events.'

7.3 Caching — Complete Reference

Cache Strategy	When to Use	Downside
Cache-Aside (Lazy)	Read-heavy. App checks cache first, loads from DB on miss	Cache miss penalty. Thundering herd problem
Write-Through	Writes go to cache AND DB simultaneously	Write latency doubles. Cache has stale-resistant data
Write-Behind (Write-Back)	Write to cache only; async persist to DB	Data loss risk if cache crashes before persistence
Read-Through	Cache transparently loads from DB on miss	Cold start problem — first request always slow
Refresh-Ahead	Proactively refresh cache before expiry	Wasted computation if data not accessed again

Cache Invalidation Strategies

- **TTL-based:** Simple, automatic. Risk of stale data. Set TTL based on acceptable staleness.
- **Event-based:** Publish cache-invalidation events on data change. Kafka or Redis pub/sub.
- **Write-through invalidation:** Update cache on write. Complex in distributed systems (cache stampede).

7.4 Database Scaling Strategies

Strategy	How It Works	Use Case	Limitation
Read Replicas	Replicate data to N replicas. Reads from replicas, writes to primary	Read-heavy workloads	Replication lag — stale reads
Vertical Scaling	Bigger machine: more CPU, RAM, faster disk	Quick fix	Hardware limit, single point of failure
Horizontal Sharding	Split data across multiple DB instances by shard key	Massive data / write scale	Cross-shard queries complex. Resharding painful
Connection Pooling	Reuse DB connections via pool (HikariCP)	High concurrent connections	Pool exhaustion under spike
CQRS	Separate read/write models and data stores	Complex query + high write load	Eventual consistency, dual-write complexity
Denormalization	Duplicate data to speed up reads	Read-heavy reporting	Write complexity, data consistency

7.5 Top 25 System Design Questions

#	System to Design	Core Concepts Tested
1	Design URL Shortener (bit.ly)	Hashing, KV store, redirect, analytics, rate limiting
2	Design Twitter/X Feed	Fan-out on write vs read, timelines, Cassandra, Redis caching
3	Design WhatsApp/Chat System	WebSockets, message ordering, delivery receipts, media storage
4	Design Uber / Ola	Geohashing, driver matching, surge pricing, real-time location
5	Design Netflix / YouTube	CDN, adaptive bitrate, blob storage, recommendations
6	Design Instagram / Photo sharing	Blob storage (S3), CDN, follower graph, feed generation
7	Design Rate Limiter	Token bucket, sliding window log, Redis atomic ops, distributed
8	Design Search Autocomplete	Trie, top-K, Redis sorted sets, offline aggregation
9	Design Notification System	Push/pull, priority queues, delivery guarantee, deduplication
10	Design Payment System (Razorpay)	Idempotency, exactly-once, saga pattern, compliance, ledger
11	Design Food Delivery (Swiggy)	Order lifecycle, real-time ETA, geo-routing, restaurant catalog
12	Design Google Drive / Dropbox	Chunked upload, deduplication, sync protocol, metadata DB
13	Design Web Crawler	BFS/DFS, politeness, deduplication, distributed coordination

14	Design Distributed Cache (Redis)	Consistent hashing, eviction policies, replication, pub/sub
15	Design Distributed Message Queue (Kafka)	Partitions, offsets, consumer groups, retention, compaction
16	Design API Gateway	Routing, auth, rate limiting, circuit breaking, logging
17	Design E-commerce Platform (Flipkart)	Catalog, inventory, cart, checkout, orders, search
18	Design Booking System (BookMyShow)	Seat reservation, payment, concurrency, overbooking prevention
19	Design Logging & Monitoring	ELK stack, metrics aggregation, alerting, distributed tracing
20	Design Distributed ID Generator	Snowflake ID, UUID, auto-increment tradeoffs, clock skew
21	Design Load Balancer	Algorithms: Round Robin, Least Connections, IP Hash, health checks
22	Design Key-Value Store	LSM trees, SSTables, bloom filter, compaction, WAL
23	Design Leaderboard System	Redis sorted sets, sliding window, real-time updates
24	Design Live Streaming Platform	RTMP ingestion, transcoding, CDN edge, WebRTC
25	Design Fraud Detection System	Real-time streaming, feature store, ML scoring, rule engine

8. Kafka & Event-Driven Architecture

Internals · Patterns · Production · Interview Q&A

8.1 Kafka Architecture — Must Know Internals

Concept	Explanation
Topic	Logical channel for messages. Divided into partitions
Partition	Ordered, immutable log. Unit of parallelism. Key → partition mapping
Offset	Monotonically increasing position within a partition
Consumer Group	Parallel consumers each assigned partition(s). One consumer per partition max
Replication Factor	Number of copies of a partition. Recommended 3 for production
Leader / Follower	One leader per partition handles all reads/writes. Followers replicate
ISR (In-Sync Replicas)	Set of replicas caught up with leader. acks=all waits for all ISR
Log Compaction	Retains latest value per key. Enables state reconstruction
Producer Acks	acks=0: fire and forget. acks=1: leader only. acks=all: all ISR
Consumer Lag	Difference between latest offset and committed offset. Monitor in production

8.2 Delivery Guarantees — Most Asked Kafka Topic

Guarantee	How Achieved	Trade-off
At-most-once	acks=0, auto-commit before processing	Can lose messages. Never use for payments
At-least-once	acks=all, commit after processing	Can duplicate messages. Must be idempotent consumer
Exactly-once	Idempotent producer (enable.idempotence=true) + Transactions + transactional consumer	Higher latency, complexity. Use for payment events

8.3 Kafka Consumer Group Rebalancing

- **Trigger:** New consumer joins, consumer crashes, topic partition count changes.
- **Stop-the-World Rebalance:** All consumers pause consumption during rebalance. Can cause lag spike in production.
- **Cooperative Rebalancing (Kafka 2.4+):** Only revoked partitions are reassigned. Other consumers continue processing.
- **Production Fix:** Set session.timeout.ms, heartbeat.interval.ms correctly. Use static membership (group.instance.id) to avoid rebalance on rolling restarts.

8.4 Kafka Interview Questions

Question	Key Answer
How does Kafka guarantee ordering?	Ordering guaranteed within a partition only. Use same partition key for related events (e.g., userId)
How do you scale Kafka consumers?	Add partitions to topic (can't reduce). Add consumers up to partition count. Beyond that, no benefit
What happens when a consumer is slow?	Consumer lag grows. If lag > retention, messages expire. Add consumers or increase retention
How do you handle poison pill messages?	Dead letter topic (DLT). After N retries, publish to <topic>.DLT. Separate consumer processes DLT
What is log compaction?	Retains last value per key. Deleted keys marked with tombstone (null value). Good for changelog topics
Difference between Kafka and RabbitMQ?	Kafka: log-based, pull model, replay, high throughput. RabbitMQ: queue-based, push, ack-delete, complex routing
How do you implement exactly-once in Kafka?	enable.idempotence=true on producer + transactional API + read_committed isolation on consumer
What is Kafka Streams?	Library for stream processing on top of Kafka. Stateful (KTable), stateless (KStream), windowing, joins

9. AWS Backend Engineering

9.1 AWS Services — Backend Engineer's Map

Service	Purpose	Key Concepts to Know	Interview Frequency
EC2	Virtual machines	Instance types (compute/memory optimized), Auto Scaling Groups, AMIs	★★★★☆
S3	Object storage	Bucket policies, presigned URLs, multipart upload, storage classes, lifecycle	★★★★★
RDS	Managed relational DB	Multi-AZ (HA), Read Replicas, connection pooling (RDS Proxy), snapshots	★★★★★
Lambda	Serverless functions	Cold start, concurrency limits, triggers (API GW, S3, DynamoDB), layers	★★★★☆
API Gateway	Managed API layer	REST vs HTTP API, throttling, caching, Lambda proxy, authorizers	★★★★☆
DynamoDB	NoSQL key-value/doc	Partition key design, GSI/LSI, DAX cache, streams, on-demand billing	★★★★☆
SQS / SNS	Queuing / pub-sub	SQS: FIFO vs Standard, DLQ, visibility timeout. SNS: fan-out pattern	★★★★☆
ElastiCache	Managed Redis/Memcached	Cluster mode, replication, failover, eviction policies	★★★★☆
ECS / EKS	Container orchestration	Task definitions, service discovery, Fargate vs EC2 launch type	★★★☆☆
CloudWatch	Observability	Metrics, logs, alarms, dashboards, structured logging	★★★★☆
IAM	Identity & access	Roles, policies, least privilege, service roles, resource policies	★★★★☆
VPC	Virtual networking	Subnets (public/private), NAT, Security Groups vs NACLs, VPC peering	★★★☆☆
Route 53	DNS + routing	Weighted, latency, failover, health checks, geo routing	★★★☆☆
CloudFront	CDN	Edge caching, origin groups, signed URLs, cache invalidation	★★★☆☆

9.2 AWS Architecture Patterns

Event-Driven Serverless Pattern

- **API Gateway → Lambda → SQS → Lambda → RDS/DynamoDB**
- Use case: Order processing, async workflows, decoupled microservices
- Key benefit: No servers to manage, auto-scales to zero, pay per invocation
- **Watch out:** Lambda concurrency limits (1000 default), cold starts for latency-sensitive APIs, 15-min max execution

High-Availability Web Application Pattern

- **Route 53 → CloudFront → ALB → ECS/EC2 (Multi-AZ) → RDS Multi-AZ + Read Replicas → ElastiCache**
- Auto Scaling Group manages EC2 fleet. RDS Multi-AZ for failover (<30s). ElastiCache for session + query cache.

9.3 Top 25 AWS Interview Questions

Question	Key Answer
Difference between SQS Standard and FIFO?	Standard: at-least-once, near-unlimited throughput, may reorder. FIFO: exactly-once, ordered, 300 TPS (3000 with batching)
What is the S3 presigned URL?	Time-limited URL signed with IAM credentials. Allows unauthenticated upload/download. Used for private content delivery
How do you handle Lambda cold starts?	Provisioned Concurrency (keeps instances warm). Minimize package size. Use Lambda SnapStart (Java 11+)
What is the difference between ALB and NLB?	ALB: Layer 7, content-based routing, WebSocket, gRPC. NLB: Layer 4, ultra-low latency, static IP, TCP/UDP
How do you make RDS highly available?	Multi-AZ: synchronous standby in another AZ. Automatic failover. Read Replicas for read scale (async, not HA)
What is DynamoDB's partition key design principle?	Choose high-cardinality key to distribute data evenly. Hot partition = throttling. Add random suffix to hot keys
How do you secure an S3 bucket?	Block public access, bucket policies, IAM roles, SSE (server-side encryption), VPC endpoints, CloudTrail logging
What is AWS IAM Role vs IAM User?	User: long-lived credentials (avoid in production). Role: temporary credentials, assumed by services/instances
How does Auto Scaling work?	Scale-out/in based on CloudWatch metrics (CPU, custom). Cooldown period prevents thrashing. Target tracking simplest
What is VPC peering vs Transit Gateway?	Peering: 1-to-1, non-transitive. Transit Gateway: hub-and-spoke, connects many VPCs and on-premises
How do you implement blue-green deployment on AWS?	Two identical environments. Route 53 / ALB weighted routing. Shift traffic gradually, rollback by changing weights
What is AWS API Gateway throttling?	Account limit 10K RPS. Per-method throttling. Usage plans with API keys. Returns 429 Too Many Requests
How do you store secrets in AWS?	AWS Secrets Manager (auto-rotation) or SSM Parameter Store (cheaper, no auto-rotation). Never in env vars or code
What is CloudWatch Logs Insights?	Query language for CloudWatch logs. Like SQL for logs. Use for debugging and metrics extraction
How do you achieve cross-region disaster recovery?	S3 cross-region replication, RDS read replicas in other region, Route 53 failover routing, global tables (DynamoDB)

10. SQL & Database Optimization

10.1 Indexing — Most Important SQL Topic

Index Type	Use Case	Downside
B-Tree (default)	Range queries, equality, ORDER BY, most general purpose	Write overhead for INSERT/UPDATE/DELETE
Hash Index	Exact equality only (=, IN). Faster than B-tree for equality	No range queries, no ORDER BY
Covering Index	Index includes all columns needed by query (no table lookup)	Larger index size
Composite Index	Multi-column index. Column order matters — follows left-prefix rule	Only usable for left-prefix queries
Partial Index	Index on subset of rows (WHERE clause in index). Smaller index	Only used when query matches condition
Full-Text Index	Text search. tokenization, stemming, relevance ranking	Not for exact equality. PostgreSQL/MySQL specific

⚡ Left-Prefix Rule: Index on (A, B, C) helps queries on A alone, (A,B), or (A,B,C). NOT on B alone or C alone. Always put highest cardinality / most filtered column first.

10.2 EXPLAIN / Query Optimization

- **EXPLAIN ANALYZE:** Shows actual execution plan with row counts and cost. Look for: seq scans on large tables, high cost nodes, nested loops on large sets.
- **Seq Scan → Index Scan:** Add index on filter/join columns. Check if query is selective enough (>10% selectivity may prefer seq scan).
- **Avoid SELECT *:** Fetches unnecessary columns. Prevents covering index usage. Always project only needed columns.
- **Avoid functions on indexed columns:** WHERE YEAR(created_at) = 2024 kills index. Use: WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'
- **JOIN optimization:** Ensure join columns are indexed. Filter early to reduce join set size. Prefer INNER JOIN over OUTER JOIN when possible.

10.3 Transaction Isolation Levels

Isolation Level	Dirty Read?	Non-Repeatable Read?	Phantom Read?	Use Case
READ UNCOMMITTED	Yes	Yes	Yes	Never use in production
READ COMMITTED (default PG)	No	Yes	Yes	Most web apps — good balance

REPEATABLE READ (default MySQL)	No	No	Yes (MySQL: No)	Reporting, consistent snapshots
SERIALIZABLE	No	No	No	Financial transactions, strict correctness

10.4 Top 25 SQL Interview Questions

Question	Key Answer
What is the difference between WHERE and HAVING?	WHERE filters rows before GROUP BY. HAVING filters groups after aggregation. Cannot use aggregate in WHERE
Difference between INNER, LEFT, RIGHT, FULL JOIN?	INNER: matching rows only. LEFT: all left + matching right. RIGHT: all right + matching left. FULL: all rows both sides
What is a covering index?	Index contains all columns needed by the query. No table lookup needed. Fastest possible index scan
How do you find duplicate rows?	SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1
What is a database deadlock? How to detect?	Two transactions each wait for a lock the other holds. SHOW ENGINE INNODB STATUS (MySQL) or pg_locks (PG)
Explain MVCC (Multi-Version Concurrency Control)	Each transaction sees a snapshot. Readers don't block writers. PostgreSQL and MySQL InnoDB use MVCC
What is a window function? Give example.	Aggregate over a partition without collapsing rows. ROW_NUMBER(), RANK(), LAG(), LEAD(), SUM() OVER(PARTITION BY)
What is the N+1 query problem?	1 query for parent + N queries for each child. Fix: JOIN FETCH, batch fetch, or denormalize
What is table partitioning?	Split large table into smaller physical pieces by range/list/hash. Partition pruning speeds queries dramatically
How do you optimize a slow query?	EXPLAIN ANALYZE → identify bottleneck → add index → rewrite query → denormalize if needed → cache result
What is the difference between TRUNCATE and DELETE?	TRUNCATE: DDL, faster, no WHERE, resets auto-increment, not easily rolled back in some DBs. DELETE: DML, supports WHERE, triggers fire
What is a composite key vs surrogate key?	Composite: multiple columns. Surrogate: single auto-generated ID. Surrogate preferred for JOINs (smaller, stable)
When would you use NoSQL over RDBMS?	NoSQL: dynamic schema, horizontal scale, KV/document patterns, high write throughput. RDBMS: relations, ACID, complex queries
What is connection pooling?	Reuse pre-established DB connections. HikariCP for Java. Prevents connection overhead. Pool size = (core count * 2) + disk spindle count
What is the difference between optimistic and pessimistic locking?	Optimistic: check version on update, retry on conflict (low contention). Pessimistic: lock row on read (SELECT FOR UPDATE), high contention

11. Revision System & Mock Interview Strategy

Spaced repetition · Mock frameworks · Company-specific prep

11.1 Spaced Repetition Schedule

Day	Review What
Day 1 (Learn)	New topic — deep study + notes + 2 coding problems
Day 2	Re-solve yesterday's problems without hints
Day 4	Quick review of notes (10 min) + 1 related problem
Day 7	Flashcard review of all concepts from the week
Day 14	Mini-test: 3 problems from 2 weeks ago, timed
Day 30	Full section revision: review cheatsheet + 5 problems

11.2 Mock Interview Framework

- **Solo Mock:** Set 45-min timer. Pull random LeetCode medium. Solve out loud. Record yourself. Review for communication gaps.
- **Peer Mock:** Take turns interviewing each other. Use real interview questions. Give structured feedback: Clarity, Approach, Code Quality, Communication.
- **Platform Mocks:** Pramp, Interviewing.io (free 1-2/month), LeetCode mock interview, HackerEarth company mocks.
- **System Design Mock:** Draw on paper/whiteboard, explain out loud for 45 min. Ask partner to probe your assumptions.

11.3 Company-Specific Preparation

Company	DSA Focus	System Design Focus	Special Topics
Amazon	Arrays, DP, Graphs, Trees	Scalable distributed systems, event-driven	Leadership Principles (STAR stories), OOP design
Flipkart	DP, Graphs, Sliding Window	E-commerce architecture, search, catalog	LLD — design patterns, SOLID principles
Razorpay/PhonePe	Data structures, concurrency	Payment gateway, idempotency, reconciliation	Fintech: exactly-once, audit trail, compliance
Swiggy/Zomato	Graphs, BFS, Greedy	Real-time tracking, geo-distributed systems	Geo-hashing, ETA computation, surge pricing
Uber	Graphs, DP, Heaps	Dispatch, maps, surge pricing, driver-rider match	Geospatial indexing, real-time stream processing
Microsoft	All patterns, OOP	Azure services, distributed systems	Design patterns, C# concepts (adaptable to Java)

Walmart	DP, Arrays, SQL	Supply chain, inventory, checkout system	Scale: millions of daily transactions, consistency

11.4 Interview Communication Guide

How to Communicate During a Coding Interview

- **Step 1 — Paraphrase:** 'So you need me to find X given Y, with constraint Z. Is that right?'
- **Step 2 — Examples:** 'Let me walk through with [1,2,3] as input. I expect output [3,2,1]...'
- **Step 3 — Brute Force:** 'A naive approach would be O(n²) — iterate all pairs. Let me think about optimization.'
- **Step 4 — Optimize:** 'I can use a HashMap to trade space for time and get this to O(n).'
- **Step 5 — Code:** Narrate as you code: 'I'm initializing the map here... now iterating... checking complement...'
- **Step 6 — Test:** 'Let me trace through with the example: map is empty, add 2, now 7 - 2 = 5, check map...' Then edge cases.

How to Answer 'Why This Approach?'

- State what alternatives you considered: 'I could use sorting (O(n log n)) but HashMap gives O(n)'
- State the tradeoff explicitly: 'We're trading O(n) space for O(n) time improvement'
- Connect to constraints: 'Given n can be 10^6, O(n²) won't scale, so O(n) is necessary'

How to Answer Tradeoff Questions in System Design

- 'We chose Cassandra over MySQL here because write throughput is 10K/s and we can accept eventual consistency for user feeds'
- 'We could use Redis for session storage (fast, ephemeral) or DynamoDB (durable, more expensive). Given we need persistence, DynamoDB is better here'
- 'The tradeoff is consistency vs latency. For user settings, we need strong consistency. For feed ranking, eventual is fine'

12. Quick Reference Cheatsheets

Last-minute revision · Interview day reference

12.1 DSA Time Complexity Cheatsheet

Algorithm / Structure	Average Case	Worst Case	Space
Array access	O(1)	O(1)	O(n)
Array search (unsorted)	O(n)	O(n)	O(1)
Array search (sorted, BS)	O(log n)	O(log n)	O(1)
HashMap get/put	O(1)	O(n) — all same bucket	O(n)
Binary Search Tree ops	O(log n)	O(n) — skewed	O(n)
Heap insert/delete	O(log n)	O(log n)	O(n)

Merge Sort	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n²) — bad pivot	O(log n)
BFS / DFS (Graph)	O(V + E)	O(V + E)	O(V)
Dijkstra (heap)	O((V+E) log V)	O((V+E) log V)	O(V)
DP (2D)	O(n * m)	O(n * m)	O(n * m)

12.2 Java Concurrency Quick Reference

Need	Use	Why
Simple counter	AtomicInteger	Lock-free, CAS-based, thread-safe
Boolean flag across threads	volatile boolean	Visibility without locking overhead
Mutual exclusion with timeout	ReentrantLock.tryLock(timeout)	Avoids deadlock risk
Multiple readers, one writer	ReentrantReadWriteLock	Readers don't block each other
Wait for N tasks to finish	CountDownLatch	countdown() + await()
Thread-local request context	ThreadLocal<T>	No sharing needed; remove() in finally
Concurrent modification safe list	CopyOnWriteArrayList	Iteration never throws CME
High-performance concurrent map	ConcurrentHashMap	Segment/bucket locking, no full lock
Async computation	CompletableFuture	Non-blocking, composable
Scheduled task	ScheduledExecutorService	Better than Timer — catches exceptions

12.3 Spring Boot Annotations Cheatsheet

Annotation	Layer	Purpose
@SpringBootApplication	Main	@Configuration + @EnableAutoConfiguration + @ComponentScan
@RestController	Web	@Controller + @ResponseBody — returns JSON
@RequestMapping / @GetMapping etc.	Web	Maps HTTP method + path to handler method
@PathVariable / @RequestParam	Web	Extracts from URL path / query string
@RequestBody / @ResponseBody	Web	Deserialize/serialize HTTP body via Jackson
@Service / @Component / @Repository	Business/Data	Stereotype annotations — mark for component scan
@Autowired / @Qualifier / @Primary	IoC	Dependency injection — by type / by name / default

@Transactional	Data	Transaction boundary. propagation, isolation, rollbackFor
@Cacheable / @CacheEvict	Cache	Enable method caching / invalidate cache
@Async	Async	Run method in thread pool. Returns CompletableFuture
@Scheduled	Task	Run on cron or fixed rate. @EnableScheduling needed
@Value / @ConfigurationProperties	Config	Inject from properties. @ConfigProps for grouped config
@ExceptionHandler / @ControllerAdvice	Error	Handle exceptions globally or per controller
@PreAuthorize / @Secured	Security	Method-level security. SpEL expression supported
@Entity / @Table / @Column	JPA	Map class to DB table. Column constraints and naming

12.4 Kafka Quick Reference

Config	Key Setting	Production Recommendation
Producer acks	acks=all	Ensures all ISR replicas acknowledge. Highest durability
Producer retries	retries=Integer.MAX_VALUE + max.in.flight=1	Enable.idempotence=true for ordering
Consumer commit	enable.auto.commit=false	Manual commit after processing — prevents data loss
Consumer poll	max.poll.records=500	Balance throughput vs processing time
Replication factor	3 (minimum for production)	Tolerates 1 broker failure
Min ISR	min.insync.replicas=2	With acks=all, need 2 ISR to confirm write
Retention	retention.ms=604800000 (7 days)	Adjust based on replay requirements
Compression	compression.type=snappy	Good CPU/compression ratio balance

12.5 Production Readiness Checklist

API Design Checklist

- ☑ Versioning: /v1/resource
- ☑ Pagination: cursor-based for large datasets (not offset — skips/duplicates with concurrent writes)
- ☑ Idempotency: Idempotency-Key header for POST/PATCH mutation APIs
- ☑ Rate Limiting: 429 Too Many Requests with Retry-After header
- ☑ Request validation: @Valid + MethodArgumentNotValidException → 400 Bad Request
- ☑ Error format: RFC 7807 Problem Details: {type, title, status, detail, instance}

- ☑ Timeouts: Set connect + read timeout on all external calls. Never use default (infinite)
- ☑ Circuit Breaker: Resilience4j on all downstream calls

Microservices Checklist

- ☑ Service Discovery: Eureka / Kubernetes DNS / AWS Cloud Map
- ☑ Distributed Tracing: trace-id + span-id propagation via HTTP headers and Kafka headers
- ☑ Centralized Config: Spring Cloud Config / AWS Parameter Store
- ☑ Health endpoints: /actuator/health for liveness + readiness probes
- ☑ Graceful shutdown: Allow in-flight requests to complete before pod termination
- ☑ Structured logging: JSON logs with trace-id, service name, environment
- ☑ DB migrations: Flyway/Liquibase — version-controlled, backward-compatible

> 🏆 FINAL MANTRA: Depth beats breadth. One well-understood system design beats five memorized ones. One pattern mastered beats ten half-learned. Quality over quantity.

13. Behavioral & Communication Excellence

STAR framework · Amazon LPs · Salary negotiation

13.1 Amazon Leadership Principles — STAR Stories to Prepare

LP	Key Story Theme to Prepare
Customer Obsession	Time you made a hard decision prioritizing user experience over tech debt or deadlines
Ownership	Went beyond your role — fixed production issue, owned end-to-end delivery
Invent & Simplify	Simplified a complex system, automated a manual process, novel solution
Are Right, A Lot	Made a data-driven decision that turned out correct despite opposition
Learn & Be Curious	Self-taught a new technology, applied it to solve a real problem
Hire & Develop the Best	Mentored a junior, helped team member grow, raised the bar
Insist on Highest Standards	Pushed back on a quick fix, insisted on proper solution
Think Big	Proposed a strategic change, re-architected a system for 10x scale
Bias for Action	Shipped something under uncertainty, calculated risk, moved fast
Frugality	Achieved more with less — cost optimization, efficient solution
Earn Trust	Delivered difficult feedback, admitted mistake, transparent under pressure
Dive Deep	Debugged complex production issue, found root cause, fixed properly
Have Backbone; Disagree & Commit	Disagreed with a decision, stated your case, then committed to team's direction
Deliver Results	Shipped under pressure, met deadline, overcame obstacles

13.2 Last-Minute Interview Day Checklist

- **Night before:** Review 5 DSA cheatsheets + 1 system design diagram + your strongest project story
- **Morning of:** Solve 1 easy problem to warm up. Don't try new hard problems — builds anxiety
- **Technical setup:** Test mic, camera, internet, editor (LeetCode/Coderpad). Have scratch paper ready
- **Mindset:** Think aloud. Interviewers want to see your process. Stuck is fine — say 'let me think through this'
- **Questions to ask:** 'What does success look like at 6 months?' / 'What's the biggest technical challenge the team faces?'